A counterexample to packing-domination inheritance in strong products

River Zhang[1] Yongxian Zhang[2]

[1]Chengdu Neusoft University, Chengdu, China
[2]School of Computer Science and Engineering,
South China University of Technology, Guangzhou, China

Corresponding author: mioqwq@hotmail.com
ORCID (R. Zhang): 0009-0004-2437-8566
ORCID (Y. Zhang): 0009-0000-3864-3536

Preprint version 1.3 — 11 October 2026

Abstract

The d-distance p-packing domination number of a graph is the minimum size of a set that dominates within distance d and separates distinct centers by distance at least $p+1$, with value infinity when no such set exists. Bujtás, Iršič Chenoweth, Klavžar and Zhang conjectured that nonexistence in one factor is inherited by every strong product containing that factor. We construct a finite connected simple graph H, with 608 vertices and 9,168 edges, for which

$$\gamma_2^3(Q_6) = \infty \qquad \text{but} \qquad \gamma_2^3(Q_6 \boxtimes H) \le 64.$$

The auxiliary graph uses singleton and distance-four pair supports in the binary six-cube. Compatibility of support unions prevents short paths between incompatible code labels, while explicit eight-label sets provide radius-two coverage at pair vertices. We give a complete proof from actual graph adjacency. A Lean formalization checks the complete existence-level counterexample on finite simple graphs and standard graph walks; two separately implemented exact checkers reconstruct and verify the finite graph and its same-center covering witness. The numerical minimum is not claimed to be 64, and historical priority is not asserted.

Keywords. Distance domination; packing; strong graph product; hypercube; explicit counterexample; formal verification.
2020 Mathematics Subject Classification. Primary 05C69; Secondary 05C76, 68V15.

Preprint status. This version is a public research preprint and has not undergone external human peer review or received journal acceptance. The construction and mathematical proof are unchanged from the reviewed v1.2 draft; this revision updates human authorship, affiliations and publication declarations. Substantial AI participation is disclosed in Section 12.

1 Introduction

Distance domination and packing impose opposing requirements on a set of graph vertices. A dominating set must place centers near all vertices, whereas a packing must keep distinct centers apart. Their simultaneous feasibility is therefore a structural question, rather than merely an

optimization problem. Following Bujtás, Iršič Chenoweth, Klavžar and Zhang [2], we write $\gamma_d^p(F)$ for the minimum cardinality of a d-distance dominating p-packing in a graph F. The value is ∞ when there is no feasible set. The underlying joint notion was introduced by Beineke and Henning [1] using the older (p, d)-domination notation; our notation follows the contemporary source.

For the strong product $G \boxtimes H$, the standard distance formula is

$$\mathrm{dist}_{G\boxtimes H}((g,h),(g',h')) = \max\{\mathrm{dist}_G(g,g'), \mathrm{dist}_H(h,h')\}.$$

One can consequently combine feasible sets from both factors by taking their Cartesian product. The converse existence question is less immediate: can the second factor supply separation missing in the first while still allowing the same product centers to dominate every layer? Conjecture 3.1 of [2] proposes the implication

$$\gamma_d^p(G) = \infty \quad \Longrightarrow \quad \gamma_d^p(G \boxtimes H) = \infty \quad \text{for every graph } H. \tag{1}$$

The hypothesis concerns only G; it does not require both factors to have infinite packing-domination number.

The source proves the conjectured implication for several classes and parameter ranges. It identifies $d < p < 2d$ as the remaining nontrivial interval. When $p = 2d$, feasible sets are perfect d-codes: their closed radius-d balls form a partition. Perfect-code inheritance in strong products is established in [4]. Our example instead uses $d = 2, p = 3$, where balls associated with a packing may overlap. This is the smallest integral parameter pair in the remaining interval. The support construction below uses that overlap directly. Thus the result concerns existence of a feasible set, before any comparison of finite optimal values. A product can restore feasibility even when one factor admits no feasible set at all.

1.1 Main result

Theorem 1.1. *There is a finite nonempty connected simple undirected graph H with 608 vertices and 9,168 edges such that*

$$\gamma_2^3(Q_6) = \infty, \qquad \gamma_2^3(Q_6 \boxtimes H) \le 64. \tag{2}$$

More precisely, H is the graph of Definition 5.1, and $\mathcal{C} = \{(a, A_a) : a \in \mathbb{F}_2^6\}$ is a 2-distance dominating 3-packing in its strong product with Q_6.

The theorem contradicts (1) with graphs that are finite, simple and connected. The original conjecture does not restrict H to a cycle, to a regular graph, or to a graph isomorphic to G. Our auxiliary factor is designed to couple the two requirements. Its adjacency is explicit; no desired distance table is postulated.

The proof separates into three tasks. First, a classical binary distance-sum argument and a small structural calculation show that no subset of Q_6 is feasible. Second, compatibility of supports excludes all paths of length at most three between code vertices whose labels are too close in Q_6. Third, each auxiliary vertex sees, within two steps, a set of code labels covering the entire cube within distance two. The final step uses one label for both coordinates; separate covering witnesses in the two factors would not suffice.

All mathematical arguments needed for the theorem are given here. Sections 9 and 10 describe additional kernel and exact-program checks without replacing the proof. The distance-sum method is classical Plotkin counting [7], and the product distance formula is standard [6]. Compatibility graphs and clique supports are also established general tools. We do not claim novelty for these ingredients or assert historical firstness for this counterexample.

2 Related work

Beineke and Henning [1] studied independent distance domination under a joint condition with separation parameter p and covering radius d. The parameter in this paper is the same simultaneous packing and domination requirement, with notation adopted from [2]. It must not be confused with the two separate optimization parameters for packing and domination. Fisher [5] studied domination, fractional domination, 2-packing and graph products. We cite that work for the older product context, without attributing to it an unverified statement about the joint parameter considered here.

Abay-Asmerom, Hammack and Taylor [4] established the inheritance of perfect r-code nonexistence in strong products. Their result applies to the endpoint $p = 2d$, where the closed dominating balls partition the vertex set. The conjecture in [2] extends this existence question to general packing-domination parameters; our counterexample lies strictly between d and $2d$. It therefore preserves the perfect-code result while disproving the proposed extension. The close-vertex hypothesis used for a positive result in [2] is also absent from our auxiliary graph, as proved in Proposition 8.2.

The companion work [3] investigates computational complexity, exact values for paths and cycles, and bounds for trees. Those results do not impose such a restriction on the universal quantifier over H in Conjecture 3.1. Our auxiliary graph need not be a cycle, a tree, or a second hypercube. The source correspondence is stated explicitly in (1), and all graphs used here are finite and connected.

The counting argument for the cube obstruction is a classical Plotkin-type method [7]; the strong-product metric is standard [6]. Neither ingredient is claimed as a new theorem. The contribution presented here is the explicit support construction and its complete verification as a counterexample to the stated universal implication. Targeted literature checks have not established historical firstness; their limitations are recorded in Section 11. Lean [8] and Mathlib [9] provide an additional, separately delimited formal check of the existence assertion, described in Section 9.

3 Definitions and graph distances

Throughout the proof, a graph is finite, nonempty, simple and undirected; graphs whose distances we use are connected. A walk is allowed to repeat vertices. The distance $\mathrm{dist}_F(u, v)$ is the least length of a u-to-v walk in F. For $r \geq 0$, let

$$B_r^F(u) = \{v \in V(F) : \mathrm{dist}_F(u, v) \leq r\}.$$

We suppress the superscript when the ambient graph is clear.

Definition 3.1. Let d, p be nonnegative integers. A set $D \subseteq V(F)$ is a *d-distance dominating set* if

$$\forall v \in V(F)\ \exists a \in D :\ \mathrm{dist}_F(v, a) \leq d.$$

It is a *p-packing* if

$$a, b \in D,\ a \neq b \quad \Longrightarrow \quad \mathrm{dist}_F(a, b) \geq p + 1.$$

The packing-domination number is

$$\gamma_d^p(F) = \begin{cases} \min\{|D| : D \text{ has both properties}\}, & \text{if a feasible set exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

The source [2] states the domination condition only for vertices outside D. This is equivalent to the closed-ball formulation above, since a center dominates itself at distance zero and $d \geq 0$. For a finite graph, existence or nonexistence of a feasible set determines whether this minimum is finite or infinite.

Definition 3.2. The strong product $G \boxtimes H$ has vertex set $V(G) \times V(H)$. Distinct vertices (g, h) and (g', h') are adjacent exactly when each coordinate is either equal or adjacent in its factor:

$$(g = g' \text{ or } gg' \in E(G)) \quad \text{and} \quad (h = h' \text{ or } hh' \in E(H)).$$

The distinctness condition excludes loops. Equivalently, a product edge moves in the first coordinate, in the second, or in both simultaneously.

Lemma 3.3. *For connected graphs G, H,*

$$\operatorname{dist}_{G \boxtimes H}((g, h), (g', h')) = \max\{\operatorname{dist}_G(g, g'), \operatorname{dist}_H(h, h')\}.$$

Proof. Project a product walk of length ℓ to either factor and remove steps during which that coordinate stays fixed. The resulting factor walk has length at most ℓ, proving the lower bound.

Conversely, take shortest factor paths of lengths r, s. Move along both at each step until the shorter path ends, then keep its endpoint fixed while completing the other path. Every step is an edge of the strong product, and the resulting walk has length $\max(r, s)$. □

Let $V = \mathbb{F}_2^6$, with coordinatewise addition. For $x \in V$, its Hamming weight $\operatorname{wt}(x)$ is the number of nonzero coordinates, and

$$\delta(x, y) = \operatorname{wt}(x + y)$$

is the Hamming distance. Simultaneous translation and permutation of coordinates preserve δ. We write 0 for the zero word, $\mathbf{1} = (1, 1, 1, 1, 1, 1)$, and $\bar{a} = a + \mathbf{1}$. The graph Q_6 joins exactly the pairs at Hamming distance one.

Lemma 3.4. *The graph Q_6 is connected and* $\operatorname{dist}_{Q_6}(x, y) = \delta(x, y)$.

Proof. Each edge flips one coordinate, so any walk from x to y must flip each differing coordinate at least once. Its length is at least $\delta(x, y)$. Flipping precisely those coordinates gives a path of that length. □

4 The obstruction in the single cube

The following proof applies to arbitrary subsets. No linearity, affine-subspace structure, prescribed cardinality or parity restriction is imposed.

Lemma 4.1. *If distinct words of $D \subseteq V$ are at Hamming distance at least four, then $|D| \leq 4$. If $|D| = 3$ or 4, every distance between distinct members of D is exactly four.*

Proof. For m binary words $x_1, \ldots, x_m$, let t_j be the number whose j-th coordinate is one. Counting pairs that differ in each coordinate gives

$$\sum_{1 \leq i < k \leq m} \delta(x_i, x_k) = \sum_{j=1}^{6} t_j(m - t_j). \tag{3}$$

If five words have pairwise distance at least four, the left side is at least $4\binom{5}{2} = 40$. On the right each contribution is at most $2 \cdot 3 = 6$, so the sum is at most 36, a contradiction. Every larger set contains five words.

For three words the upper bound from (3) is $6 \cdot 2 = 12$, exactly the lower bound $4\binom{3}{2}$. For four words the bounds are both $6 \cdot 4 = 24 = 4\binom{4}{2}$. Equality of the sum and the pairwise lower bounds forces every pair distance to be four. □

Lemma 4.2. *Translate a three- or four-word distance-four packing so that it contains* 0*. Every other word has weight four, and its two zero coordinates form a pair. These zero pairs are mutually disjoint. For four centers, the three zero pairs partition the six coordinates.*

Proof. The first assertion follows from Lemma 4.1. If b, c have weight four and zero sets Z_b, Z_c of size two, then

$$\delta(b, c) = |Z_b \triangle Z_c| = 4 - 2|Z_b \cap Z_c|.$$

This equals four precisely when the zero sets are disjoint. Three disjoint two-element sets exhaust the six coordinates. □

Lemma 4.3. *A radius-two ball in Q_6 has 22 vertices. The radius-two balls about two distance-four centers intersect in six vertices. For three pairwise distance-four centers their three balls have a unique common vertex. For four such centers the fourfold intersection is empty.*

Proof. The first count is $1 + 6 + \binom{6}{2} = 22$. For two centers translate one to 0 and call the other b, with $\mathrm{wt}(b) = 4$. If x is in both balls, then

$$4 = \delta(0, b) \le \delta(0, x) + \delta(x, b) \le 4.$$

Equality forces both distances to be two and x to be supported on two of the four nonzero coordinates of b. Conversely every such x is in the intersection. There are $\binom{4}{2} = 6$.

For three centers, translate them to $0, b, c$. Their two disjoint zero pairs leave two coordinates R outside $Z_b \cup Z_c$. A word in all three balls must be a weight-two word supported on the intersection of the supports of b, c, which is precisely R. Thus the word with support R is the unique common vertex.

For four centers $0, b, c, e$, the third zero pair is $Z_e = R$. The word supported on R differs from e in all six coordinates, so it is not in the fourth ball. The same argument applies after choosing any of the centers as the translated zero; every triple therefore has exactly one common vertex. □

Proposition 4.4. *Every Hamming-distance-four packing $D \subseteq V$ has a vertex at distance greater than two from every member of D. Consequently $\gamma_2^3(Q_6) = \infty$.*

Proof. Lemma 4.1 gives at most four members. With zero, one or two centers, the union of their radius-two balls has size at most 44, less than 64.

For three centers Lemma 4.3 and inclusion–exclusion give

$$\left| \bigcup_{a \in D} B_2^{Q_6}(a) \right| = 3 \cdot 22 - 3 \cdot 6 + 1 = 49.$$

For four centers the corresponding count is

$$\left| \bigcup_{a \in D} B_2^{Q_6}(a) \right| = 4 \cdot 22 - 6 \cdot 6 + 4 \cdot 1 = 56.$$

Neither union covers V. Lemma 3.4 identifies these Hamming balls with graph balls and proves the obstruction. □

Remark 4.5. There is also a direct witness for three or four centers. Complete the zero pairs of Lemma 4.2 to a partition into three pairs. A word choosing exactly one coordinate from each pair is at distance three from 0 and from every weight-four center in the packing. The eight such words are all omitted. This explains the obstruction without the union counts, though the counts specify the total covered size.

5 A graph built from compatible supports

Define the set of unordered distance-four label pairs by

$$\mathcal{P}_4 = \{\{a, b\} \subseteq V : \delta(a, b) = 4\}.$$

Every pair has two distinct members. Each label has $\binom{6}{4} = 15$ labels at distance four, so

$$|\mathcal{P}_4| = \frac{64 \cdot 15}{2} = 480. \tag{4}$$

Definition 5.1. The vertices of H form the disjoint tagged union

$$\{A_a : a \in V\} \sqcup \{S_a : a \in V\} \sqcup \{E_P : P \in \mathcal{P}_4\}.$$

We call A_a a code vertex. The other vertices are called Steiner vertices and have supports

$$\sigma(S_a) = \{a\}, \qquad \sigma(E_P) = P.$$

The edges of H are precisely these unordered pairs:

1. $A_a T$, where T is a Steiner vertex and $a \in \sigma(T)$;
2. TU, where T, U are distinct Steiner vertices and every pair of distinct words in $\sigma(T) \cup \sigma(U)$ has Hamming distance at least four.

There are no code–code edges, no loops and no other edges.

The support union in the second clause is a set. A shared word is counted once and is never compared with itself. In particular S_a may be adjacent to $E_{\{a,b\}}$. Tags distinguish code vertices from singleton Steiner vertices despite their shared label. Adjacency is symmetric and excludes equality, defining an ordinary simple undirected graph.

Proposition 5.2. *The graph H has 608 vertices and is connected.*

Proof. Equation (4) gives $64+64+480 = 608$ vertices. For a coordinate word e_i, there is a two-step path

$$S_a - S_{\bar{a}} - S_{a+e_i},$$

because the consecutive label distances are six and five. Thus every single-coordinate flip is realizable between singleton vertices. Repeating such paths connects all singleton vertices. Each A_a is adjacent to S_a, and each $E_{\{a,b\}}$ is adjacent to A_a, A_b. Every vertex belongs to the same component. □

Lemma 5.3. *If $a \neq b$ and $\operatorname{dist}_H(A_a, A_b) \leq 3$, then $\delta(a, b) \geq 4$.*

Proof. A path of length one is impossible because no code–code edge exists. A path of length two has the form $A_a - T - A_b$, with both labels in $\sigma(T)$. A singleton cannot support two distinct labels, so $T = E_{\{a,b\}}$ and $\delta(a, b) = 4$.

A path of length three has the form $A_a - T - U - A_b$ with both internal vertices Steiner: an internal code vertex would create a code–code edge at one of the ends. The edge TU makes distinct words in their support union separated by at least four. The union contains a, b, giving the conclusion. These cases include shared-endpoint supports and exhaust all paths of positive length at most three. □

Define the candidate centers by

$$\mathcal{C} = \{(a, A_a) : a \in V\}. \tag{5}$$

There are exactly 64 distinct product vertices in this set.

Proposition 5.4. *The set $\mathcal{C}$ is a 3-packing in $Q_6 \boxtimes H$.*

Proof. Consider distinct labels a, b. If $\delta(a, b) \geq 4$, the first coordinate separates their centers by at least four. Otherwise Lemma 5.3 and connectedness give $\mathrm{dist}_H(A_a, A_b) \geq 4$. Lemma 3.3 proves separation at least four in either case. □

6 Explicit covering sets of labels

For $h \in V(H)$, put

$$L(h) = \{a \in V : \mathrm{dist}_H(h, A_a) \leq 2\}.$$

Product coverage follows if each $L(h)$ radius-two covers the cube. We first establish the two label facts needed for this claim.

Lemma 6.1. *For every $a \in V$, the set*

$$L_a = \{a\} \cup \{b \in V : \delta(a, b) = 4\}$$

radius-two covers V.

Proof. Translate a to zero. A word of weight zero, one or two is covered by zero. A weight-three word extends to a weight-four word at distance one. A weight-four word is itself a center. A weight-five word contains a weight-four word at distance one, and a weight-six word is at distance two from any weight-four word. These cases account for every word in V. □

Lemma 6.2. *For $a, b \in V$ with $\delta(a, b) = 4$, define*

$$K(a, b) = \{a, b\} \cup \{c \in V : \delta(c, a) \geq 4,\ \delta(c, b) \geq 4\}.$$

This is an eight-element set, and it radius-two covers V.

Proof. Translate a to zero and permute coordinates so $b = (1111, 00)$. Write $c = (r, s')$, with four-bit first block r and two-bit second block s'; put $t = \mathrm{wt}(r)$, $s = \mathrm{wt}(s')$. The two common-far-label inequalities become

$$t + s \geq 4, \qquad 4 - t + s \geq 4.$$

They imply $s \geq t$ and $s \geq 4 - t$. Since $s \leq 2$, necessarily $t = s = 2$. Conversely these block weights satisfy both inequalities. Hence

$$K(0, b) = \{(0000, 00), (1111, 00)\} \cup \{(u, 11) : u \in \mathbb{F}_2^4,\ \mathrm{wt}(u) = 2\}. \tag{6}$$

The six latter words are distinct from the endpoints.

For an arbitrary target $x = (r, s')$ with block weights t, s, the endpoint distances are $t + s$ and $4 - t + s$. Among the six other labels the minimum distance is

$$\min_{\mathrm{wt}(u)=2} \delta(x, (u, 11)) = |t - 2| + 2 - s. \tag{7}$$

The overlap of $\mathrm{supp}(u)$ with $\mathrm{supp}(r)$ can be $\min(t, 2)$, giving first-block distance $t+2-2\min(t, 2) = |t - 2|$. Such a two-element support exists for every $0 \leq t \leq 4$, and the second-block distance to 11 is $2 - s$.

It remains to check

$$\min\{t + s,\ 4 - t + s,\ |t - 2| + 2 - s\} \leq 2. \tag{8}$$

For $t = 0, 4$, use the appropriate endpoint. For $t = 1, 3$, use an endpoint when $s \leq 1$, and the third expression when $s = 2$. For $t = 2$, the third expression is $2 - s \leq 2$. Thus every target is covered. □

The minimum in (8) is displayed below. The proof realizes each value, so this table is exact.

$t\backslash s$	0	1	2
0	0	1	2
1	1	2	1
2	2	1	0
3	1	2	1
4	0	1	2

Proposition 6.3. *For every auxiliary vertex h, the set $L(h)$ radius-two covers V.*

Proof. We treat all three types in Definition 5.1. For $h = A_a$, the label a belongs to $L(h)$. Each b at distance four from a also belongs, via

$$A_a - E_{\{a,b\}} - A_b.$$

Thus $L_a \subseteq L(A_a)$, and Lemma 6.1 proves the claim.

For $h = S_a$, the edge $S_a A_a$ reaches label a. For every b at distance four from a, the union $\{a, b\}$ is compatible, giving

$$S_a - S_b - A_b.$$

Consequently $L_a \subseteq L(S_a)$, and the same lemma applies.

For $h = E_{\{a,b\}}$, both endpoint code vertices are adjacent to h. A common far label c is distinct from a, b. The support union $\{a, b, c\}$ is separated by at least four, giving

$$E_{\{a,b\}} - S_c - A_c.$$

Therefore $K(a, b) \subseteq L(E_{\{a,b\}})$. Lemma 6.2 completes the final case. □

7 The product witness and the main theorem

Proposition 7.1. *The set $\mathcal{C}$ of (5) is a 2-distance dominating set in $Q_6 \boxtimes H$.*

Proof. Fix an arbitrary product vertex (x, h). Proposition 6.3 supplies one label $a \in L(h)$ with $\delta(x, a) \le 2$. For that same label, $\mathrm{dist}_H(h, A_a) \le 2$. Hence

$$\mathrm{dist}_{Q_6 \boxtimes H}((x, h), (a, A_a)) = \max\{\delta(x, a), \mathrm{dist}_H(h, A_a)\} \le 2.$$

The center (a, A_a) belongs to $\mathcal{C}$. The argument covers every x and every type of h, thus all $64 \cdot 608 = 38,912$ product vertices. □

Proof of Theorem 1.1. Proposition 4.4 proves $\gamma_2^3(Q_6) = \infty$. Proposition 5.2 proves that H is connected and has 608 vertices. Propositions 5.4 and 7.1 give the feasible set $\mathcal{C}$ of size 64. Thus the product packing-domination number is finite and at most 64. The edge count is proved in Proposition 8.1 below. □

Remark 7.2. The first-coordinate projection of $\mathcal{C}$ is the whole cube, which is not a distance-four packing. This is compatible with product packing because the second coordinate separates every incompatible pair of labels. Its radius-two neighborhoods still contain enough labels to cover the cube. Projecting a product witness to one factor therefore does not preserve packing.

8 Edge count and the close-vertex condition

Proposition 8.1. *The graph H has exactly 9,168 edges.*

Proof. Partition the edges by vertex type. There are 64 code–singleton incidences and $2 \cdot 480$ code–pair incidences, giving 1,024. For singleton–singleton edges a fixed label has $\binom{6}{4} + \binom{6}{5} + \binom{6}{6} = 22$ compatible distinct labels. Division by two gives $64 \cdot 22/2 = 704$.

A fixed pair $\{a, b\}$ is compatible with singleton supports at a, b and at the six common far labels of Lemma 6.2. These are exactly the compatible singleton supports, giving $480 \cdot 8 = 3,840$ such edges.

For pair–pair edges sharing an endpoint, translate it to zero. The other two labels have weight four, and compatibility means their zero pairs are disjoint. There are 15 choices of the first zero pair and $\binom{4}{2} = 6$ disjoint choices of the second. Division by two gives 45 edges at each shared endpoint, hence $64 \cdot 45 = 2,880$. The shared endpoint is unique.

For disjoint pair supports, their union is a four-word distance-four packing. By Lemma 4.2, a packing containing a specified label is determined by a partition into three unordered zero pairs. There are

$$\frac{6!}{(2!)^3\, 3!} = 15$$

such partitions. Conversely, each partition gives three weight-four words whose zero pairs are its blocks; together with zero they form a valid four-word distance-four packing. Choosing a specified label in 64 ways counts every packing four times, giving $64 \cdot 15/4 = 240$ packings. Each has three partitions into two unordered pair supports, giving $240 \cdot 3 = 720$ edges. The total is

$$1,024 + 704 + 3,840 + 2,880 + 720 = 9,168.$$

□

Theorem 3.3 of [2] proves the conjectured implication when the *other* factor H has a (d, p)-close vertex: a vertex h such that all distances between vertices in $B_d^H(h)$ are at most p. The next observation checks consistency with that result.

Proposition 8.2. *Every radius-two ball of H has diameter at least four. In particular H has no $(2, 3)$-close vertex.*

Proof. For $h = A_a$ or S_a, choose two labels in L_a whose translated weight-four supports share three coordinates, for example $\{1, 2, 3, 4\}$ and $\{1, 2, 3, 5\}$. The labels have Hamming distance two and both code vertices lie in $B_2^H(h)$.

For $h = E_{\{a,b\}}$, normalize as in Lemma 6.2. Choose the labels $(u, 11), (v, 11)$ with first-block supports $\{1, 2\}, \{1, 3\}$. Again their Hamming distance is two and both code vertices belong to $B_2^H(h)$.

Lemma 5.3 puts their H-distance at least four, while their paths through h put it at most four. Thus each ball contains a pair at distance exactly four. □

9 Formal verification

The complete existence assertion has been checked in Lean [8] using Mathlib [9]. The public sources and source-specific receipts accompany this paper [10]. The environment is pinned to Lean 4.34.1 and Mathlib commit

`d13f23b723b8a846827a245b89c10fc7d3f11612`.

The dependency manifest is supplied in the isolated package used by the replay script.

9.1 Objects and the path bridge

Cube vertices have type `BitVec 6`. Hamming distance counts all six differing bits and returns a natural number; there is no bit-vector overflow in the distance result. The actual `SimpleGraph` G has adjacency $\delta(a, b) = 1$. Auxiliary vertices form a tagged sum of 64 code labels, 64 singleton labels and canonical unordered distance-four pairs. Strict numerical ordering selects one representative of every unordered pair. The source defines the symmetric loopless adjacency of H using support membership and compatibility. It proves its product adjacency equivalent to the three ordinary strong-product edge cases. No abstract distance table is assumed.

For convenient bounded lengths the formalization uses padded reachability. Write $\mathrm{Step}_F(u, v)$ for equality or adjacency in F, and define

$$\begin{aligned} \mathrm{Reach}_F(0, u, v) &\iff u = v, \\ \mathrm{Reach}_F(n+1, u, v) &\iff \exists w,\ \mathrm{Step}_F(u, w) \wedge \mathrm{Reach}_F(n, w, v). \end{aligned}$$

The theorem `reach_iff_walk` proves equivalence with existence of a standard Mathlib F-walk of length at most n. Equality steps pad a shorter walk; they do not add graph loops. The proof converts padded reachability to a walk and pads a shorter walk in the converse direction.

The theorem `G_reach_iff_hamming` proves

$$\mathrm{Reach}_G(n, a, b) \iff \delta(a, b) \le n$$

for every n. The product bridge `strong_reach` proves

$$\mathrm{Reach}_{G \boxtimes H}(n, (x, h), (a, k)) \iff \mathrm{Reach}_G(n, x, a) \wedge \mathrm{Reach}_H(n, h, k).$$

The same endpoint (a, k) appears throughout, matching the usual strong-product distance formula.

9.2 The formal endpoint and its interpretation

The definition `WalkPacking F S` states that no standard walk of length at most three joins distinct members of S. The definition `WalkCovers2 F S` states that every vertex has a standard walk of length at most two to some member of S. On connected graphs these are exactly our packing and domination properties.

The declaration `original_existence_counterexample` has the mathematical content

$$\neg\exists S \subseteq V(G),\ \mathrm{WalkPacking}(G, S) \wedge \mathrm{WalkCovers2}(G, S), \tag{9}$$

$$\exists S \subseteq V(G) \times V(H),\ \mathrm{WalkPacking}(G \boxtimes H, S) \wedge \mathrm{WalkCovers2}(G \boxtimes H, S). \tag{10}$$

The second witness is the defined set $\mathcal{C}$. The first quantifies over every cube-vertex set, not just linear codes or finite samples. The endpoint has no unproved geometric premise, external-certificate hypothesis or extra center assumption.

On finite graphs, these statements are the existence content of infinite and finite packing-domination numbers. A separate numerical function γ_d^p is not defined in the Lean sources. The written proof gives the explicit cardinality $|\mathcal{C}| = 64$, hence the bound in Theorem 1.1. No numerical minimization or optimality claim is imported into the formal endpoint.

9.3 Proof structure, audits and remaining boundaries

There are three project modules. The existing factor module proves a missing-vertex theorem for arbitrary subsets of six-bit Hamming space. It normalizes a nonempty set by XOR translation, lists the 22 possible nonzero labels at distance at least four from zero, and constructs exact missing-vertex witnesses. Ordinary `decide` proves finite certificate lemmas, including a 22^3 compatibility certificate. No power set of 64 vertices is enumerated. This is an alternative formal proof of Proposition 4.4, rather than a formalization of its inclusion–exclusion text.

The new geometry module proves Hamming metric facts, a one-bit distance-reducing step and both label-covering facts. It uses XOR normalization and ordinary kernel-checked `decide` on small closed finite statements. The graph module supplies actual graphs, bounded-walk bridges, short-center exclusion, same-center coverage and the final conjunction.

Primary records contain 12 printed axiom audits for the geometry module and 26 for the graph module. A separately run replay builds the factor, geometry and graph modules from their actual sources in a fresh project object directory, in dependency order. It records 12, 12 and 26 audits respectively. All recorded runs have exit code zero, no warnings or errors and no nonstandard axioms. Audit records inspect declarations; they are not a count of newly discovered mathematical theorems.

The final endpoint uses only the standard Lean axioms `propext`, `Classical.choice` and `Quot.sound`. The sources use no `sorry`, `native_decide`, added axiom, solver oracle or Python certificate as a proof. Public records retain source hashes, UTC intervals, logs and every printed dependency list.

The cardinality 608 of the auxiliary vertex type is a Lean theorem. Connectedness of H, its 9,168-edge count and its exact diameter are not separate Lean endpoints in this version. Connectedness and the edge count have complete written proofs above and exact graph-program checks. A numerical `SimpleGraph.dist` definition is not used; ordinary shortest paths are represented by standard walks and lengths. These boundaries do not weaken the formal existence assertion, but they must not be hidden by claiming that every auxiliary statistic has been formalized.

10 Exact computational verification

Two separately implemented graph checkers use Python's standard library and exact integer and set operations. Neither uses a floating-point optimization or solver. The mathematical proof of Theorem 1.1 does not depend on their output.

10.1 First construction and witness certificate

The first checker represents a cube word as an integer from 0 to 63 and computes Hamming distance by XOR and population count. It constructs cube edges and verifies that breadth-first search distances equal Hamming distances for all cube vertices. It constructs all 608 tagged auxiliary vertices and every edge of Definition 5.1.

Breadth-first searches from all 64 code vertices give distances to every auxiliary vertex. All $\binom{64}{2} = 2,016$ center pairs are tested using the maximum factor distance. For each of the 38,912 product targets the program saves one label satisfying both distance bounds and rechecks both bounds for every saved same-center witness. The public graph and certificate files contain the tagged adjacency data and the distance/witness arrays.

10.2 Separate reconstruction and shortest-distance checks

The second checker initially reads no graph or certificate from the first implementation. It reconstructs the vertices and support-union adjacency directly from the definitions, then compares adjacency with a separately expressed predicate on all $\binom{608}{2} = 184,528$ unordered distinct vertex pairs. It checks for missing and spurious edges.

It runs breadth-first search from all 608 vertices, obtaining 369,664 distances. These also undergo a shortest-distance certificate check: for every source v, the recorded function f_v is zero at v, positive elsewhere, differs by at most one across every edge, and each non-source vertex w has a neighbor of value $f_v(w) - 1$. The edge bound makes every v-to-w walk at least $f_v(w)$ steps long; successive descending neighbors construct a walk of exactly that length. These conditions certify shortest distances, rather than merely the shape of a BFS array. Symmetry is checked too.

The checker independently tests all 2,016 center pairs and 38,912 same-center targets. It verifies that the sufficient label sets of Section 6 lie in the actual radius-two neighborhoods and cover the cube. Only after passing this phase does it read the first checker's artifacts. The comparison matches all 608 canonical vertices, 9,168 edges, 38,912 code-distance entries and all covering witnesses.

The recorded independent run used Python 3.12.14 on 64-bit Windows. The scripts require Python 3.10 or later. There are no recorded uncovered targets or packing violations, and the minimum distance between the specified product centers is four. The complete BFS also records diameter four. That statistic is supplementary: the proof needs only connectedness, short-path exclusion and radius-two coverage.

10.3 Meaning of separate checks

Separate implementations, reconstruction before reading the first artifacts and a fresh Lean object directory reduce some common checking errors. They do not establish independent human peer review, originality or journal acceptance. AI agents participated in implementation and review. Exact-program outputs are finite evidence, the written argument is a complete mathematical proof, and the Lean endpoint is a kernel-checked existence assertion. These are related but distinct forms of evidence.

11 Discussion and limits

The obstruction in Q_6 is not inherited by every product containing it. The auxiliary graph supplies a separation coordinate while retaining the requirement that a single center dominate both coordinates. At a pair Steiner vertex, the endpoints alone do not cover the cube. The six additional labels in (6) fill the missing coverage while remaining compatible with the pair support. This eight-label calculation enables the global construction.

The theorem refutes the exact universal implication of Conjecture 3.1 in [2]. It does not contradict perfect-code inheritance at $p = 2d$ [4]: there the balls are disjoint and each target has a unique dominating center. Our balls may overlap. Nor does it contradict the close-vertex theorem; Proposition 8.2 shows that its H-side hypothesis is absent. The complexity, cycle and tree results of [3] concern other assertions.

We do not determine $\gamma_2^3(Q_6 \boxtimes H)$ exactly, minimize the number of auxiliary vertices or classify all factors repairing the cube's feasibility obstruction. The 64 centers are a witness, not an asserted optimum. Replacing H with another copy of Q_6 is a different problem: this paper does not decide whether $\gamma_2^3(Q_6 \boxtimes Q_6)$ is finite.

The primary preprint and published article were checked on 9 and 10 October 2026; their conjecture statements agree in the relevant scope. Targeted identifier, author, older (p, d)-domination and support-compatibility searches did not locate a verified earlier complete result subsuming this construction. Those searches are not exhaustive and do not establish firstness. Earlier product bounds such as [5] also involve packing and domination. A full historical audit remains necessary before asserting priority. Correctness of the explicit counterexample and historical priority are separate questions. The accompanying publication audit records the checked sources, the limits of the citation search and the precise formalization boundary. It is not a bibliometric similarity certificate or an external referee report. Before submission, the eventual human authors must examine the proof, the cited literature and the accountability requirements of the selected venue.

12 AI assistance, authorship and declarations

Substantial generative-AI assistance was used in this research. The work used OpenAI Codex with a GPT-6-based agent system; no more specific model version is asserted here. AI systems participated in proposing the support-based graph, developing and checking derivations, writing Lean sources, implementing exact graph checkers, inspecting statements and axiom lists, conducting targeted literature searches, and drafting and revising this manuscript. Separate AI-agent mathematical reviews and program reconstructions were performed; they are not described as independent human reviews. The Lean kernel checks and actual program executions are recorded separately from AI-generated reasoning.

The human coauthors are River Zhang and Yongxian Zhang, with River Zhang serving as the corresponding author. Both authors have reviewed and approved this preprint for public release and accept responsibility for its content, including the mathematical claims, referenced evidence and AI-use disclosure. Yongxian Zhang contributed software to the research effort. No designation of equal-first authorship is intended, and no AI system is listed as an author.

Funding. The authors declare that this research received no external funding.

Competing interests. Both authors declare that they have no competing interests.

License. Copyright © 2026 River Zhang and Yongxian Zhang. Both authors authorize publication of this manuscript (PDF and LaTeX source) under the Creative Commons Attribution 4.0 International License (CC BY 4.0): `https://creativecommons.org/licenses/by/4.0/`. Software, Lean

code, certificates and third-party material remain subject to the applicable separate licenses, as documented in the reproducibility archive.

Academic status. This is a non-peer-reviewed public preprint, not a journal article. Neither historical priority nor comprehensive proof of novelty is asserted. DOI information, if assigned by an external repository, is maintained in the corresponding repository record. The authors' approval of public release does not constitute independent human peer review or an assertion of journal acceptance.

13 Data and code availability

The explicit 608-vertex auxiliary graph, 64-center certificate, independently implemented Python checkers, Lean proofs and source-specific verification records are publicly available in the fixed source archive. The mathematical witness is also preserved in the immutable GitHub release. A version-matched reproducibility archive, with explicit software and manuscript license boundaries, a file-hash manifest and exact replay instructions, is available in the publication/006-v1.3 directory. The permanent Git commit that publishes the archive and its file hashes is the version-specific provenance record. Future journal-specific supplementary formats may be prepared without changing the underlying mathematical witness.

A Source correspondence and reproducibility

The repository [10] supplies the following material. The factor path is relative to the repository root. Other paths in the table are relative to `006-strong-product-packing-counterexample/`. They identify public source files rather than a private machine environment.

Mathematical content	Public source and verification boundary
Arbitrary-set cube obstruction	`notes/proof/Q6PackingDomination.lean`. The missing-vertex theorem covers every subset of `BitVec 6`.
Hamming geometry and label coverage	`proof/Q6PackingComplexGeometry.lean`. The declarations `single_cover` and `pair_cover` give actual translated covering witnesses.
Graphs, walks and complete endpoint	`proof/Q6PackingComplexCounterexample.lean`. Its `original_existence_counterexample` is the conjunction (9)–(10).
Compilation and axiom evidence	`results/lean-verification.json` and source-specific logs. The fresh rebuild compiles all three project sources.
Explicit graph and witness	`results/graph.json` and `results/certificate.json`.
Exact reconstruction and comparison	The scripts `verify_graph.py` and `verify_independent.py` in the topic directory, with graph and comparison receipts under `results/`.
Connectedness and 9,168 edges	Propositions 5.2 and 8.1, plus exact graph replay; not separate Lean theorems here.
Numerical bound and limitations	The injective label map gives $\lvert\mathcal{C}\rvert = 64$ in the written proof. The Lean endpoint defines no numerical γ_d^p function and supplies no optimality theorem.

For a fresh replay, install the pinned Lean toolchain and prepare the dependency package `002-weighted-rectangular-pruning/proof/mathlib`. Its manifest fixes Mathlib and the transitive dependencies. Obtain the required dependency artifacts using Lake's cache command if they are absent. In a fresh checkout, first run `lake update` and `lake exe cache get` from that dependency package; its Git requirement fixes the Mathlib revision. The versioned supplementary archive includes the toolchain, Git dependency configuration, transitive manifest and the external factor module, rather than requiring a private local dependency path. The replay first queries the actual Lean version and Mathlib Git revision and checks them against the pins. An environment-only check does not count as a proof compilation. The replay runs all three project sources serially and creates new object files; it does not accept old project objects as a substitute for source compilation.

Run the following commands from the repository root, with new output directories. Relative paths below are examples usable on Windows or Unix-like systems.

```
python 006-strong-product-packing-counterexample/verify_graph.py \
  q6-primary-new
python 006-strong-product-packing-counterexample/verify_independent.py \
  own --out q6-independent-new
python 006-strong-product-packing-counterexample/verify_independent.py \
  compare --out q6-independent-new --root q6-primary-new
python 006-strong-product-packing-counterexample/verify_lean.py \
  --out q6-lean-new
```

The displayed backslash continuations are Unix-like shell syntax. In PowerShell put each command on one line or use that shell's continuation syntax. The option `-lake` accepts an explicit Lake executable path.

The receipts retain source and artifact hashes and distinguish reconstruction without the first checker's data from subsequent comparison. A regenerated result should be assessed against the actual source revision and precise checked assertions. An old result file or matching timestamp is not a fresh execution.

References

[1] L. W. Beineke and M. A. Henning, *Some extremal results on independent distance domination in graphs*, Ars Combinatoria **37** (1994), 223–233. `https://combinatorialpress.com/article/ars/Volume%20037/volume-37-paper-20.pdf`.

[2] C. Bujtás, V. Iršič Chenoweth, S. Klavžar and G. Zhang, *On d-distance p-packing domination number in strong products*, Annals of Combinatorics (2026), published 26 March 2026. doi:10.1007/s00026-026-00814-0. Original preprint: arXiv:2510.02749v1, 3 October 2025.

[3] C. Bujtás, V. Iršič Chenoweth, S. Klavžar and G. Zhang, *The d-distance p-packing domination number: complexity, cycles, and trees*, Aequationes Mathematicae **100** (2026), article 25. doi:10.1007/s00010-026-01266-w.

[4] G. Abay-Asmerom, R. H. Hammack and D. T. Taylor, *Perfect r-codes in strong products of graphs*, Bulletin of the Institute of Combinatorics and its Applications **55** (2009), 66–72. `https://richardhammack.github.io/reprints/rperfect_codes_strong_prod.pdf`.

[5] D. C. Fisher, *Domination, fractional domination, 2-packing, and graph products*, SIAM Journal on Discrete Mathematics **7** (1994), no. 3, 493–498. doi:10.1137/S0895480191217806.

[6] R. Hammack, W. Imrich and S. Klavžar, *Handbook of Product Graphs*, second edition, CRC Press, Boca Raton, 2011.

[7] M. Plotkin, *Binary codes with specified minimum distance*, IRE Transactions on Information Theory **6** (1960), no. 4, 445–450. doi:10.1109/TIT.1960.1057584.

[8] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in *Automated Deduction—CADE 28*, Lecture Notes in Computer Science **12699**, Springer, 2021, 625–635. doi:10.1007/978-3-030-79876-5_37.

[9] The mathlib Community, *The Lean mathematical library*, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, ACM, 2020, 367–381. doi:10.1145/3372885.3373824.

[10] Accompanying mathematical sources and verification records, *006: strong-product packing counterexample*, `https://github.com/mio-qwq/math/tree/613108f1fa107a88a1049ed5f25e6b9313d576ed/006-strong-product-packing-counterexample`. Public source baseline for this draft: commit `613108f` (9 October 2026).